Critical review: Scale in the current state

This is the honest state of the project after the Polylith conversion and the Nu node. The review is deliberately critical: the strengths are real, and so are the gaps.

What is real

- A large F# implementation with 1530 example checks (1488 examples and 42 properties), 33 granular cases, 91 discovered tests, a differential corpus, snapshots, and a kernel boundary suite. The first test batch adds a lowering differential property, a prose-forms property, and a lowered-IR snapshot (D148).
- A NativeAOT kernel with a C ABI, image validation and swap, primitive wire, and a C adapter with a conformance harness. The adapter refuses a different ABI before resolving any other symbol; the image loader refuses a different schema.
- A Polylith workspace: `kernel`, `core`, `typing`, `prelude`, `verification`, `demo`, `cli`, `shell-host`, `book-host`, `nu-host`, and `adapter-c`; `development/scale`; the `scale-cli` base; the `scale` and `game` projects. `just poly-check` enforces boundaries and cycles.
- Typst documentation: specification, decisions, ledger, roadmap, invariants, line counts, and this review.

What is not real yet

- The human surface has a first parser slice, not the full surface. The current language is still mostly the IR/stack surface; the inline comma-delimited form (`match 5, when 1 then "one", otherwise "many".`) and the line form now parse to `ExpressionAst.PatternMatch` with lowercase keywords, spaced names and `of` phrases, text with spaces, `close`-delimited nested matches, and a sentence sequence that becomes a `Block`, with thirty-three checks (D142–D146). The sequence lowers to the kernel `LoweredAbstraction` and the final stack top is the result. `scale --prose <path>` reads and runs a `.prose` file (D149). The `run`/`capture` effects landed as host words in Stage 1a (D183); books and spells landed as `.book` packages with `--book`, `--cast`, and the prose `cast` word (D184). The `with` connective and the REPL remain the later D147 work. The "one abstraction layer above F#" is still the largest gap between the project's claim and its code.
- The Scale Prelude is not ported. The F# Prelude remains the reference; the kernel can execute image units, but the image is not populated with the real Prelude. The `docs/prelude/` feature files and the port have not started.
- Nu is a vendored node and a host skeleton. The engine is cloned (`components/nu-engine`), but no game loop loads the kernel, no image swap runs under the engine, and no AOT game target is tested. The C ABI seam is the right design; the integration is future.
- The demo content root is fixed (D141), but the release payload is still unproven. `demo.fs` now resolves `components/demo/demo` in a checkout and the copied `<base>/demo` directory in an isolated payload, and the suite reads its baselines through the same resolver. The remaining risk is the release AOT smoke, which alpha does not run: until it runs from a copied payload, the packaged demo path is code-reviewed, not executed.
- The component boundary is not perfectly clean. `components/typing` imports Hedgehog because the type lattice does; a test package sits in a production component. The evaluator's runtime guard needs the lattice, so the split is honest, but a test-free lattice core is the better target.
- The deployable used to ship the test suite (D23), and the base referenced `development/scale`. D176 split the suite into the JIT-only `projects/scale-tests` deployable; the product no longer references the test code or its packages.
- `components/cli` exists because the development suite tests the argument parser, and a development node cannot reference a base. It avoids a cycle, but the test placement, not the parser, drove the component.

- The C conformance native run is now an alpha gate. `just kernel-aot` publishes the kernel as a NativeAOT shared library and runs the full harness against it; all eleven checks pass (D180). A path-filtered CI workflow runs the same gate when the kernel or adapter changes.
- jj is installed and the repository is colocated. The `Scale-code` workspace is a sparse checkout that excludes `components/nu-engine`; it is the verified Serena/F# workspace for Scale-only work. The main `Scale` workspace remains the full checkout for Nu (D182). The LSP still breaks in the full checkout because the F# project graph includes Nu; use `Scale-code` for Scale work.

EDN and the specification

EDN is excellent for the machine-readable contract: a feature registry, the status matrix, decision ids, test mappings, conformance vectors, and the workspace manifest. `workspace.edn` already uses it. It is not a document format: it has no normative prose, no examples with explanation, no layout, and no rendered table. The right split is Typst for the normative specification and EDN for the data the documentation and checks consume. A `spec/features.edn` (feature, status, decision, tests, surfaces) with `just spec-check` validating the Typst status matrix and the suite against it would make the specification machine-checkable without losing its prose. Converting the whole specification to EDN would trade readability for a data structure the specification does not need.

Recommendations, in order

1. Implement the rest of the prose surface from the documented design (`docs/prose-surface.typ`, D142): text with spaces, nested matches, and the program result. The first parser slice is landed.
2. Run the release AOT smoke from a copied payload; the demo content root is fixed (D141), so this is now the packaged-path proof.
3. Move the lattice/Hedgehog dependency out of the production typing path or split a test-free typing core.
4. Port the Prelude feature by feature; the kernel/image path is ready.
5. Wire Nu: a minimal game base that loads an image and swaps it, then an AOT target test. Keep the C ABI as the seam.
6. Add the EDN feature registry and `just spec-check`; keep Typst normative.
7. Use the verified `Scale-code` jj workspace for Scale-only editing and keep Nu in the full main checkout (D182).
8. Split the test suite out of the deployable; it is now the JIT-only `projects/scale-tests` artifact (D176).

Verdict

A serious, well-tested, well-documented prototype with a credible native boundary and a clean component layout. It is not a finished language: the human surface, the real Prelude, and the engine integration are the work that remains. The foundation is strong enough to carry them.

Review addendum, 2026-11-04

This addendum records a full-project review. It is deliberately about the things the suite cannot see: build freshness, semantic authority, scaling ceilings, component coherence, the trust boundary, and the documentation and gate process.

The incident: a green suite against a broken HEAD

At the start of the review, the committed tree did not compile. `components/prelude/evaluator/lowering.fs` opened `TypeInference`, whose `Type` case `Variable` shadowed `PatternAst.Variable`; the newly added P4c test also used a qualified record field that F# rejects. The example suite was green only because it ran a build from several commits earlier. Two defects fixed this: the inference

references are now qualified, and the source-range expectation in `tests.fs` matches the parser (the expression range excludes the sentence period).

The durable lesson is the gate, not the two edits. A suite result is evidence only when the build it runs is the build the files describe. `just freshness-check` now compares every project's sources with its Debug output, `just docs-check` pins the facts around the binaries, and the CI and release workflows run both. SageFs already reports `sourceState: Stale`; the review found that report being ignored, not missing.

Architectural risks the tests cannot reach

1. **Two semantic authorities.** The interpreter dispatches words through a hard-coded `match` of roughly 150 names, while the image VM executes lowered units. The differential corpus compares them on 18 token programs and the generated property covers arithmetic; neither covers the dispatcher's word surface. The internal P4 signature registry covers seven primitives (`add`, `subtract`, `multiply`, `divide`, `lessThan`, `greaterThan`, `equals`), so `--prose-typecheck` refuses most real programs as unknown words. The review did not unify the engines. It did add D174′s single word index: every dispatcher word is classified, `primitiveTable` is derived from that list, and `just docs-check` fails if the two sets diverge. The remaining risk is duplicate execution semantics, not a silent signature drift.
2. **The event history was an always-on quadratic ceiling.** `Evaluator.run` recorded a step for every token: the token, the whole stack, and the whole remaining program, then appended to a vector by copying the entire history. The review made recording opt-in (`history-on`/`history-off`, default off) and replaced the vector with a persistent previous-step chain, so normal evaluation records nothing and append is O(1) when enabled (D178). The remaining cost is that a recorded step still encodes the whole stack and remaining program; a compact encoding is a separate decision.
3. **`readTextAt` scans the whole text cache on every call.** `Store` keeps `TextCache: Map<string, EntryIdentity>` and inverts it by scanning for the identity. It is used by word lookup, closures, dispatch, conditions, sets, and patterns. Dispatch therefore degraded as the image grew. The review added `TextIdentities: Set<EntryIdentity>` to the store, maintained by `addText` and rebuilt from the persisted text cache; the wire format is unchanged and `readTextAt` is now a set lookup plus a lexicon read.
4. **The cyclic evaluation state is held together by convention.** An `Actor` contains a full `EvaluationState`, which contains the `SchedulerState`, which contains every actor. The live state is canonical only because `run` threads it; an actor snapshot may carry a stale scheduler, and `initial` builds a dummy scheduler to break the cycle. The review's stage 1 moved the canonical root construction into `Scheduler.initializeRoot`, but the record cycle remains. Stages 2 and 3 of D179 move the scheduler field and callers, then remove the dummy construction.
5. **Four kinds of type coexist without one cross-reference.** The elaborator's first-order `TypeInference`, the verification lattice (`TypeLattice`), the image-resident set-theoretic sort record, and the document field types (`ScaleFile`/`DomainExpression`) use different representations and different words for type. Each is justified in its own document; the risk is a reader or a new feature choosing the wrong one. The review moved `TypeInference` to `components/typing`, moved the shared source-range types to `ExpressionAst`, and added `docs/type-map.typ`, the single cross-reference for all five notions. The representations remain intentionally distinct.
6. **The ABI was process-global behind a handle API.** `abi.fs` stored one callback pointer, one handle value, and one loaded image in module-level mutable state; a second `scale_init` replaced the first. The review replaced that with a thread-safe registry of independent handles, each with its own callback table and image, and reset the ABI to version 1 because no host had used the old

contract (D175). The header states the per-handle lifetime and concurrency rules; a stale handle is invalid and freeing it is a no-op.

7. **The persistence trust boundary was inconsistent.** The `CorruptStore` comment claimed that a malformed file becomes a named failure rather than an arbitrary exception, while truncation raised `EndOfStreamException` and `BinaryReader.ReadString` allocated from an unchecked prefix. The review now bounds text by `maxTextBytes` before allocation, uses a checked 7-bit length decoder, and converts truncation to `CorruptImageException`. Semantic and truncation corruption are one named failure, and the suite pins it.
8. **The vendored engine dominates repository navigation.** A retail Nu engine is checked in under `components/nu-engine` (4,245 tracked files). It is an external Polylith node and is excluded from line counts, but it dominates search, agent context, and clone size. This is a known cost of D140, not a defect; the root `.ignore` now keeps it out of ignore-aware searches, and the line-count tool already excludes it.

Process and documentation findings

The operational layer had drifted from the code: `AGENTS.md` still said ABI version 1 and expected 1148 checks and a 78-case granular suite, the `justfile` expected 1300 checks, `just doctor` listed `*.fsproj` at the repository root (always empty), the docs index omitted roughly half the Typst corpus, and five non-canonical Markdown/text notes lived under `docs/`. The review fixed these: the legacy notes moved to `Notes/`, the index is complete, `AGENTS.md` carries the current ABI, counts, and granular writer checks, and `just docs-check` fails on any recurrence.

The gate was weaker than its name. CI ran formatting, build, and `--all-tests`, but not the Polylith boundary, the C adapter compile, the documentation checks, or the nine JIT-only writer checks; the release workflow did the same and lacked `dotnet tool restore`. Both workflows now run the architecture, docs, conformance, freshness, and kernel AOT checks and the granular suite. `just check` runs the same set, including `kernel-aot` (D180). The remaining environment coupling is deliberate but worth naming: `just check` begins with `mcp-check`, which reads the user's OpenCode configuration and requires a live SageFs daemon, so the one documented gate cannot run in CI; CI mirrors it with the hermetic subset.

The repository has a pinned SDK (`global.json`), central package management (`Directory.Packages.props`), and a dual license: BSD-2-Clause-Patent for code (`LICENSE`) and CC0-1.0 for documentation (`LICENSE-docs`).

What the review changed

1. Fixed the compile break and the wrong source-range expectation; the example suite is 1530 checks (1488 examples, 42 properties), the granular suite is 33 cases (11 snapshots, 13 integrations, 9 writer checks), and both are green.
2. Moved `type-inference.fs` from `core` to `typing` and the source-range types to `ExpressionAst`, removing the parser-to-inference dependency.
3. Added `tools/consistency-check.fsx` (`just docs-check`) and `tools/freshness-check.fsx` (`just freshness-check`), a pinned granular count, truthful summary totals that include failures, and the CI/release gate steps.
4. Added the store's `TextIdentities` index, so `readTextAt` no longer scans the whole text cache on every dispatch, and replaced the process-global ABI state with independent handles; the ABI is reset to version 1.
5. Added a checked, bounded persistence reader: truncated streams and oversized text lengths are `CorruptImageException` with a named reason.

6. Added D174 and `PreludeSignatures.classification`: one built-in word index, derived `primitiveTable`, and a `docs-check` ratchet against the dispatcher.
7. Added `docs/type-map.typ`, the cross-reference for the five type-like notions.
8. Added `global.json`, `Directory.Packages.props`, and a root `.ignore` for the vendored Nu engine; added the BSD-2-Clause-Patent / CC0-1.0 dual license.
9. Split the suite into the JIT-only `projects/scale-tests` deployable; the product base no longer references `development` (D176).
10. Made event history opt-in with a persistent previous-step chain (D178) and
11. moved the canonical root construction to `Scheduler.initializeRoot`
12. (D179 stage 1).
13. Added the kernel NativeAOT + C conformance gate (D180).
14. Colocated the repository with jj and added the sparse `Scale-code` workspace,
15. which excludes `components/nu-engine` so the F# project graph contains only
16. Scale projects (D182).
17. Updated `AGENTS.md`, `justfile`, `docs/README.typ`, the decision-range
18. reference, and the component path references.

The open architectural items above are the agenda after the prototype work: one execution authority, a compact history-step encoding, and the remaining D179 stages for the evaluator/scheduler split.